Pipeline des formulaires
&
gestion mémoire

Réalisé par Aicha Nait Dayaa
Encadré par Abdelmajid Bendrif

Drupal Form API

Pipeline & Workflow

Introduction

Lorsqu'on crée un formulaire, on utilise le Form API. Ce dernier est le système complet qui gère la construction, l'affichage, la validation et la soumission des formulaires.

Au cœur de ce système se trouve le service FormBuilder, qui agit comme un orchestrateur du pipeline : il exécute les différentes étapes dans le bon ordre et déclenche les bons handlers au bon moment.

Le Pipeline Complet

Étape 1 - Point d'entrée : `getForm()`

Tout commence par `FormBuilder::getForm()`. C'est le seul point d'entrée pour tout formulaire. FormBuilder instancie la classe du formulaire, crée un objet `$form_state` et y stocke cette instance via `$form_state->getFormObject()`. Ce mécanisme est ce qui permet au système de retrouver et d'appeler les méthodes du formulaire tout au long du pipeline.

Étape 2 - Le formulaire est-il en cache ?

Dès le départ, FormBuilder vérifie si le formulaire existe déjà en cache.

- **OUI** : il récupère le `$form` et le `$form_state` depuis le cache afin de conserver l'état du formulaire (valeurs saisies, erreurs, étape courante) et permettre la continuité du traitement.
- **NON** : il appelle la fonction de build pour construire le formulaire.

Étape 3 - Construction : `buildForm()`

FormBuilder fait appel à la méthode `buildForm()` en lui passant un `$form_state` vide lors du premier affichage, ou un `$form_state` existant lors d'une reconstruction. Vous construisez et retournez le tableau `$form` avec tous vos éléments.

Si votre `buildForm()` retourne directement une **Response** — comme une redirection ou une erreur — FormBuilder la retourne immédiatement et le pipeline s'arrête là.

Étape 4 - Préparation : `prepareForm()`

FormBuilder appelle `prepareForm()` qui prépare le formulaire pour son utilisation. Elle génère un `build_id` unique - Cet identifiant permet d'identifier et de retrouver ce formulaire dans le cache lors de soumissions ultérieures - ajoute le token CSRF pour les utilisateurs authentifiés, et enregistre les méthodes comme handlers dans le tableau `$form`. e.g. :

```
$form['#validate'][] = '::validateForm';
$form['#submit'][]   = '::submitForm';
```

Elle déclenche également les hooks d'altération `hook_form_alter()`, `hook_form_BASE_FORM_ID_alter()` et `hook_form_FORM_ID_alter()` pour permettre aux modules de modifier le formulaire.

Étape 5 - Processus de build récursif

FormBuilder parcourt récursivement chaque élément du formulaire de haut en bas pour initialiser les valeurs de `$form_state`, collecter les valeurs saisies via les `#value_callback`, détecter le bouton déclencheur, exécuter les callbacks `#process` et `#after_build`, et déterminer les handlers de validation et de soumission.

Étape 6 - Traitement : `processForm()`

C'est ici que le pipeline se sépare selon le type de requête, contrôlé par `$form_state->isProcessingInput()`.

- **Requête GET** : `isProcessingInput()` retourne `FALSE`. Le formulaire est affiché. `validateForm()`.
- **Requête POST** : `isProcessingInput()` retourne `TRUE`. Le traitement complet s'enclenche.

Étape 7 - Vérification du token CSRF

Première chose que fait `processForm()` lors d'une soumission POST : vérifier le token de sécurité. Si le token est invalide, le message *"The form has become outdated"* est déclenché et le pipeline s'arrête immédiatement.

Étape 8 - Validation : `validateForm()`

La validation suit un processus récursif de bas en haut. Pour chaque élément, le système exécute automatiquement les validations core (`required`, `maxlength`, options valides) et les callbacks `#element_validate`.

Ensuite, FormBuilder délègue à **FormValidator** qui lit `$form[#validate]` enregistré dans `prepareForm()`, récupère l'instance via `$form_state->getFormObject()` et appelle la méthode `validateForm()`.

> Si le bouton cliqué possède ses propres handlers `#validate`, ils **remplacent** votre `validateForm()`.

Étape 9 - Des erreurs existent-elles ?

Après la validation, `processForm()` consulte `FormState::hasAnyErrors()`.

- **OUI** : `submitForm()` est **ignorée**. Le formulaire est reconstruit et renvoyé à l'utilisateur avec les messages d'erreur et les valeurs préservées.
- **NON** : le pipeline continue vers la soumission.

Étape 10 - Soumission : `submitForm()`

FormBuilder délègue à **FormSubmitter** qui lit `$form[#submit]`, récupère l'instance via `$form_state->getFormObject()` et appelle la méthode `submitForm()`.

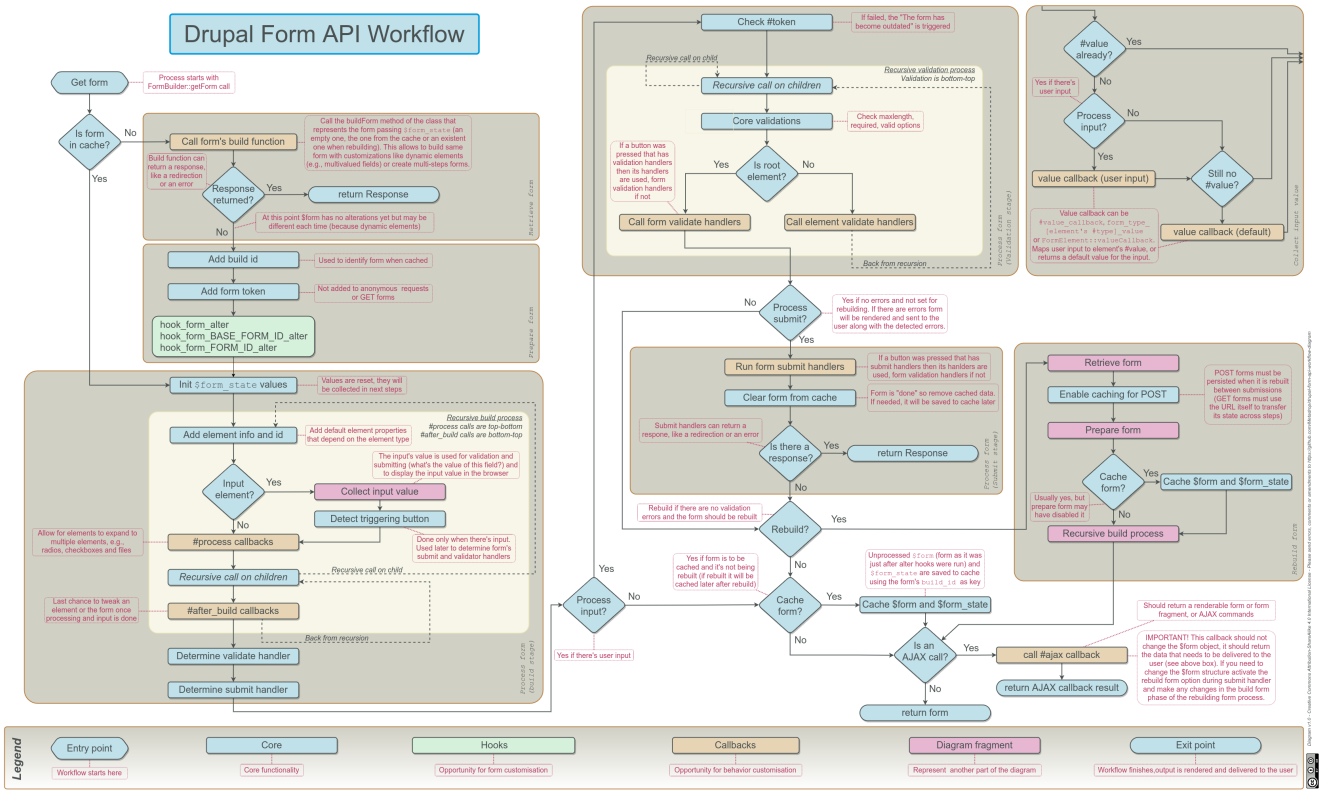
> Si le bouton cliqué possède ses propres handlers `#submit`, ils **remplacent** votre `submitForm()`.

Étape 11 - Réponse, Reconstruction ou Retour final

Condition	Résultat
`$form_state->setRedirect()` appelé	Redirection immédiate
`$form_state->setRebuild(TRUE)` appelé	Retour à `buildForm()`
Requête AJAX	Exécution du `#ajax callback`

Le diagramme officiel de Drupal.org illustre avec plus de détails ce pipeline dans son intégralité.

Form Api workflow diagram



PHP Memory Limit

Configuration & Bonnes Pratiques

Introduction

Lorsqu'un script PHP s'exécute, il consomme de la mémoire RAM pour stocker variables, objets, tableaux et résultats de requêtes. La directive `memory_limit` définit la quantité maximale de mémoire qu'un seul processus peut utiliser. Si un script dépasse cette limite, PHP lance une erreur fatale :

```
Fatal error: Allowed memory size of X bytes exhausted
```

C'est avant tout une mesure de **protection du serveur**, sans cette limite, un seul script pourrait consommer toute la RAM disponible et provoquer le crash de l'ensemble des services.

Les Niveaux de Configuration et leur Priorité

`memory_limit` peut être définie et surchargée à plusieurs niveaux. Chaque niveau écrase le précédent, du moins prioritaire au plus prioritaire.

Niveau 1 : `php.ini` (Configuration globale)

Fichier de configuration principal de PHP. S'applique à **tous les scripts** du serveur, sauf en cas de surcharge.

```
memory_limit = 128M
```

Niveau 2 : `.htaccess` (Surcharge par répertoire)

Permet de surcharger `memory_limit` pour un répertoire spécifique.

```
# .htaccess
php_value memory_limit 256M
```

Niveau 3 : `settings.php`

Surcharge la valeur pour toute l'application via `ini_set()`.

```
// web/sites/default/settings.php
ini_set('memory_limit', '256M');
```

Niveau 4 : `ini_set()`

Modification à la volée dans un script spécifique.

```
ini_set('memory_limit', '512M');
```

Niveau 5 : Par commande CLI

Lors de l'exécution d'une commande en ligne de commande, il est possible de surcharger `memory_limit` directement sans modifier aucun fichier de configuration.

```
php -d memory_limit=512M script.php
```

C'est la méthode recommandée lorsqu'on a besoin temporairement de plus de mémoire, plutôt que de modifier `settings.php` ou `php.ini` et risquer d'impacter l'ensemble de l'application.

Pourquoi Augmenter `memory_limit` est une Mauvaise Pratique

Lorsqu'on atteint la limite mémoire, le réflexe immédiat est d'augmenter la valeur. C'est une **mauvaise pratique** pour deux raisons essentielles.

C'est traiter le symptôme, pas la cause. Une erreur mémoire est un signal d'alarme indiquant un problème dans le code ; requête chargeant trop de données, boucle accumulant des objets, module mal optimisé. Augmenter la limite revient à éviter l'alarme sans résoudre le problème.

Cela masque des fuites mémoire.

Que Faire Quand On Atteint `memory_limit` ?

1. Identifier la source avant d'agir

```
echo memory_get_usage(true);        // memoire utilisee actuellement
echo memory_get_peak_usage(true);   // pic de consommation atteint
```

2. Optimiser les requêtes

3. Utiliser le Batch Processing

4. Augmenter en dernier recours